Extron ControlScript Design

Pieter Nauwelaerts & Larry Ma

UCSC Media Systems Engineering

Contents

1 Introduction

The Extron ControlScript extension for VS Code makes system integration easier by allowing Extron Authorized Programmers to create complex control systems through the use of the Python scripting language. While Extron provides resources on how to use the extension and its related utilities, they may be difficult for those without prior Python experience to understand.

This document compiles various tips and tricks regarding how UCSC's systems are currently set up (as of school year 2024-25) and how systems can be expanded or changed depending on the hardware contained in each room. It also discusses graphical layouts and how to implement functionality for them using Python. This document applies specifically to Extron Touch Panels. Network Button Panels (NBPs) or eBUS panels work in a slightly different way to the customizable screens used in large classrooms.

Our hope is to provide any MSE employee with the fundamental knowledge and resources necessary to operate and maintain UCSC's ControlScript codebase. If there is anything else unclear - Google is your best friend!

2 Setting up the ControlScript Environment

Extron makes ControlScript accessible as an extension for Microsoft's "Visual Studio Code". This allows programmers to develop ControlScript platforms in a powerful and easy-to-use environment. Installing the following should be sufficient to get a ControlScript environment up and running.

2.1 Installing Visual Studio Code

The VSCode installer can be found at: https://code.visualstudio.com/download. Any version greater than 1.67.0 will suffice.

2.2 Installing Python

The Python installer can be found at: https://www.python.org/downloads/release/python-354/. Extron insists that users install Python 3.5, as it is the "most compatible" with Extron's processors. The current codebase runs Python 3.12.2

2.3 Installing the Control Script Extension

The ControlScript Extension can be found at:
https://www.extron.com/product/software/controlscriptvscode.

The download should yield a .VSIX file that can be installed in VSCode in 3 steps:

1. Click on the "Extensions" menu on the left sidebar.
2. Click on the "Views and More Actions" meatball menu.
3. Click on "Install from VSIX", and select the Control Script VSIX file from its local location.

Refer to the figure below for further help.

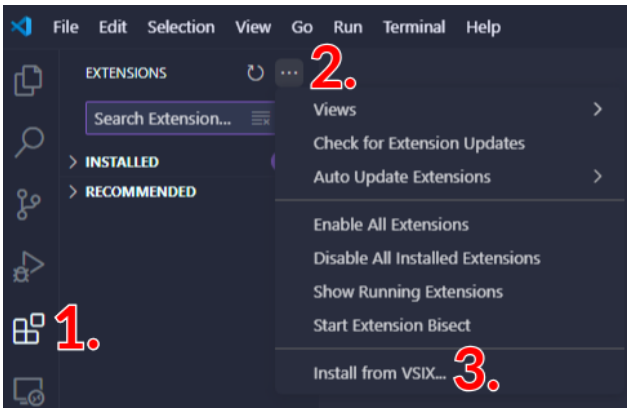

Figure 1: Installing the ControlScript Extension in VSCode

2.4 Installing the ControlScript Deployment Utility

This tool can be used to deploy and debug system programs and load or retrieve projects from control processors. The ControlScript Deployment Utility Installer can be found at: https://www.extron.com/product/software/controlscriptutility.

3 Python Fundamentals & Resources

ControlScript requires a familiar understanding of basic Python. This section contains a list of free resources related to the Python programming language.

For those **without** programming experience:

- Automate the Boring Stuff (Easy to follow, online book - we recommend starting here): https://automatetheboringstuff.com/#toc
- Python Beginner Tutorials (Youtube Playlist): https://youtube.com/playlist?list=PL-osiE80TeTskrapNbzXhwoFUiLCjGgY7&si=UO9nMBcGZAkFolxp
- Intro to Programming course from the University of Helsinki (Online, self-paced course complete w/ videos + exercises): https://programming-23.mooc.fi/
- Harvard's CS50: Introduction to Computer Science (Online, self-paced course from Harvard University): https://pll.harvard.edu/course/cs50-introduction-computer-science
- CodingBat (coding problems - a personal favorite!): https://codingbat.com/python

For those **with** programming experience:

- The Python Tutorial (Official): https://docs.python.org/3/tutorial/
- Learn X in Y minutes (Python Guide): https://learnxinyminutes.com/docs/python/

3.1 A Bare Minimum Understanding

"Automate the Boring Stuff" does an outstanding job of introducing and explaining basic Python concepts. Below is a list of recommended ATBS Chapters and the relevant topics that they cover.

- **Chapter 1: Python Basics**
  - Math operators
  - Data types
  - Variables: assignment and naming
- **Chapter 2: Flow Control**
  - Boolean Values
  - Comparison Operators
  - And, Or, Not operators
  - Flow Control Statements: If, else, elif, while, for
  - Importing Modules
- **Chapter 3: Functions**
  - Defining, Calling
  - Arguments and Parameters
  - Return Values + Statements
- **Chapter 4: Lists**
  - Indexing
  - Length
  - Concatenation, slicing

Understanding these first four chapters of ATBS will provide the basic knowledge sufficient to navigate the ControlScript codebase; It is recommended that you read through all chapters entirely.

4 Project File Structure & Basics of Design

Control script projects can be created in two different flavors. Both methods use Extron templates included in the VS Code extension. For instructions on how to create a project, it is best to follow the instructions provided by Extron. By using the VS Code Command Palette, opened with `ctrl + shift + p`, you can create a new project through the extension. The instructions will ask you for everything needed to setup a new project and devices can be added at that point or later. Device selection is covered in more detail in Section 7.

The template requires selecting a control processor and a UI device in order to begin. Device setup like IP addresses are not necessary until you start deploying the project and can be left as the defaults. Device Aliases are used to link the code and json so it is best to name it something memorable and distinct. Each name must be unique.

Once you have a template project created you can go one of two ways with it. The first is to write all of your code in `src/main.py` which might work for smaller projects, but larger ones containing more than just a few buttons, pages, or device integrations, should follow the second design philosophy, that being using multiple files for specific functions meant to do. we use the second design approach and we think for any room in the university, it is the only way you should be designing. The following section will detail in depth the structure of the template and the responsibilities of each file.

4.1 UI Layouts and Code Design

The template layout creates multiple folders and Python script files for you the designer to fill in with any device or instruction you need. The base folder, named after whatever you put in the Project Name field of the template, contains two folders and a file `<Project Name>.json` The JSON file contains configuration information for your included devices such as their device number, alias, and IP address. These fields need to be correct otherwise deployment will cause issues or devices will not connect properly.

The two folders contained are layout and src. Layout contains only the GUI Designer file you will be displaying on your touch panel. If you have multiple non-mirrored panels, their GUIs go here. The JSON has an entry for assigning which GUI goes to which panel. This path and file name also needs to be copied into the project JSON for configuration and deployment. Src is the more complicated of the two in this file structure and can be broken into the next 4 sections

4.1.1 Control

The control folder contains code written for building or AV control purposes. Say you want a button on the panel that dims the lights in the room, this is where you put that Button event. The framework doc is unclear whether this file is meant to contain ALL events for AV equipment such as audio controls, so in my implementation we put control commands to devices in these files, and UI responses in the TLP files.

4.1.2 Modules

The module folder contains two subfolders. Helper contains Extron created helper files such as `ModuleSupport.py` or `ConnectionHandler.py` which provide methods to support your project creation. Classes and decorators like `@event()` are contained in this folder and information on their use will be described in Section 3. ConnectionHandler contains the code used to connect your devices together. Creating a new device instance requires a connection using one of the methods provided in this file. Device connections will be discussed in Section 4.

4.1.3 UI

UI contains the code for each touch panel used in the system. If you have more than one touch panel in use, you can change how the layout interacts by defining which Python code corresponds to which touch panel object. `tlp.py` contains all UI objects and Events needed to control the system. Further information will be provided in Section 3.

4.1.4 src/ Files

Four files are contained in the `~/src` directory. `devices.py` is meant to contain your device definitions. This file is used to set up all of the different devices the system is responsible for controlling. `main.py` is the main file responsible for initializing the system and starting everything up. It is the file targeted by the deployment utility. This can be changed in the project JSON but is not reccomended. `system.py` contains the connections to any networked devices using the `ConnectionHandler.py` file methods to connect them. `variables.py` contains any system variables you want to define for use throughout the files. Things like audio levels could be used here so that they are easier to update. Any variable that will need to be used in many files should be placed here.

4.1.5 Design Consistency

One of the most important ideas to remember when writing or changing code for these devices is to label everything consistently. If you are creating a Button, have the button name include or somehow refer to what that button is doing. For example, my HDMI source selection button could be called something like `btn_HDMI`. In my code we follow this naming scheme:

- `btn_<name>` A Button with name referring to the function of the UI element.
- `sld_<name>` A Slider with a name referring to the function of the UI element
- `dv<NAME>` A device created in `src/devices.py`. Most devices have a python module related to them contained in `src/modules/devices` containing functions described in their communication sheet. More on this in Section 7.1
- `Lbl<name>` A Label element with name referring to the function of the UI element. Used for visual feedback in passcode screens or audio levels.
- `<name>_set` An MESet of elements, usually buttons, which allows only one button of the set to be selected at any time. This includes for visual feedback and

Various UI elements can be defined and used by the code to create functionality for the touch panel. we use these naming conventions to stay consistent and to show the difference between UI types. Calling something btn_something but defining a slider is not a good idea when other people are trying to interpret your code.

5 Initializing a ControlScript Project

In order to initialize a project there is only one requirement. You only need to have an Extron control processor. Only certain processors are qualified and the list of them can be found by using the "Create New Project" option in the command palette. Once you select this option you must choose a project template. The default Extron template is what all of our current projects have been built on. We have considered creating a template for each room type but have not yet made one.

5.1 Choosing a Processor

Once you have selected the Extron Default Template you will be asked what control processor you are using. This depends on the needs of the space as well as what other devices you are choosing to use. The processor choice is based on communication port needs, compatible touch panels or physical button panels, combination switchers, and AVLAN needs. In order to view compatibility and features, review the information sheets specific to the processors listed here: https://www.extron.com/IPCP-Pro-Series/prodsubtype-599. Once you have selected the IPCP, give it an IP address if you know it, otherwise edit it later inside the project JSON. You are also able to use touch panels with Link License instead of selecting an IPCP or an IPL from the list of processors.

5.2 JSON Entries

This section will cover project settings in the JSON. More information about device entries to the JSON files are covered in Section 7.2.1.

The JSON file is the collection point for your folder structure. Following the default template will create a JSON file with the following structure:

```
{
    "system": {
        "name": "<Name entered at init>",
        "code_folder_path": "src",
        "code_entry_file": "main.py",
        "layout_folder_path": "layout",
        "sound_folder_path": "sound",
        "rfile_folder_path": "rfile",
        "irfile_folder_path": "ir",
        "system_id": "2928",
        "data_file_path": "data.json",
        "author": {
            "name": "Your Name",
            "email": "Your Email"
        },
        "version": "1.0.0",
        "primary_device_alias": "ProcessorAlias"
    },
    "devices": [
        {
            "name": "Primary Processor",
            "part_number": "60-1431-01",
            "alias": "ProcessorAlias",
            "network": {
                "host_network_type": "LAN",
                "interfaces": [
                    {
                        "address": "192.168.254.254",
                        "type": "LAN"
                    },
                    {
                        "address": "192.168.254.254",
```

```
33                      "type": "AVLAN"
34                  }
35              ]
36          }
37      }
38  ],
39  "schemaVersion": "1.0.0"
40  }
```

To add a touch panel, simply add a comma at line 33, add a newline, then type TLP and select your model from the list of devices. The entry that is shown will look something like this:

```
1  {
2      "name": "Primary Touch Panel",
3      "part_number": "60-1565-02",
4      "alias": "PanelAlias",
5      "extron_control_web_id": "5f4a4d67-5447-4378-a860-1c69d2c38a9d",
6      "network": {
7          "host_network_type": "LAN",
8          "interfaces": [
9              {
10                 "address": "192.168.254.254",
11                 "type": "LAN"
12             }
13         ]
14     },
15     "ui": {
16         "layout_file": "Project1.gdl",
17         "enable_click_sound": True,
18         "press_feedback": "none",
19         "thumb_change_report_interval": 0.1,
20         "thumb_to_fill_sync_time": 0.5
21     }
22 }
```

Depending on if your device is using LAN or AV LAN you will want to change the host network type on line 7. If you want other devices to use the LAN connection for the IPCP, make sure to set the host network type to LAN. Otherwise it needs to be on AVLAN to communicate. If you choose LAN, then both AVLAN and LAN devices are able to communicate. You must also change the layout file path to your corresponding `.gdl` file name to get the proper layout file. This file should be stored in the folder corresponding to `"layout_folder_path"` on line 6 of the first snippet.

You are allowed to change folder paths or names as needed. For deployed code we removed any unused folders from the top of the JSON and from the project directory to remove unnecessary clutter from the space.

At this point you now have the template for a project. Each python file is empty, so creating functionality from the GUI layout and utilizing devices will be covered in the next two sections.

6 Programming the Panel

The upside to Extron's ControlScript Extension is that python knowledge is not mandatory to understand basic touch panel programs. The only things we would recommend you understand before trying to write your own project or changing significant portions of this code will be the things listed below. We will provide code snippets while describing what they do and how to use them.

6.1 Creating UI Elements

UI elements create the interaction between the user and the technology. They make it possible to use the touch panel to do what it needs to do. In order to create functionality for the device with these UI elements, you need to have a GUI Designer file contained in the layout folder. IDs, page names, and states are needed in the python code to create UI objects. UI objects have different properties and are defined differently depending on what they do. The following sections will cover how to create UI objects in `tlp.py`

6.1.1 Buttons

To declare a Button, you need a few things. First, you need to import the proper library, then declare a variable to hold the Button class in, you need a touch panel device registered in the project and declared with a name, and finally you need the ID from the GUI Designer file. Here is how to declare a Button in python.

```
1 #at top of file only once
2 from extronlib.ui import Button
3
4 <button 1 name> = Button(<touch panel name>, <GUI ID>)
5 <button 2 name> = Button(<same touch panel name>, <different GUI ID>)
```

Creating buttons requires a name for them, and a declaration of a new `Button()` class. This class has different methods and properties we will not be listing here but a few are used in the completed code. The most used method is the `SetState()` method to set the visual feedback state of any button. Here is how that is used:

```
1 #On state (default of 1)
2 <button name>.SetState(1)
3
4 #Off state (default of 0)
5 <button name>.SetState(0)
```

There are other methods that change the text, or change other properties of the button instance. There are also parameters which return the current value of things like the visual state, name, or whether the button is being pressed or held. Methods need parentheses to be called in python but parameters do not. Here's an example:

```
1 #method - sets the visual text on the button to "Button Name"
2 #this is why we use variables because there is no way to get this text back unless it is stored in
      a variable
3 <button name>.SetText('Button Name')
4
5 #parameter - button_name is now whatever the OBJECT name is (determined in GUI designer)
6 button_name = <button name>.Name
```

Parameters return certain types. Some return true or false, some return string values, and some return integer values, all depending on what the parameter is responsible for. These methods, parameters, and return types, are well documented in the Extron ControlScript Extension Documentation that comes with the installation of the extension. Refer to this constantly when writing your code otherwise you will break something and be confused by it.

Buttons are integral for making things happen in the code. Because of the number of things these panels have to do, there are going to be a lot of buttons defined within the code. These buttons have to have something assigned to them to make them function. This functionality will be discussed in Section 6.2

6.1.2 Sliders

Sliders are visual feedback objects in the GUI designer and Global Configurator that allow a sliding scale of change to a device. Sliders are used to change the audio levels in the room. Instructors have the mic and program audio buttons to control their levels but hidden in a technician access panel is the audio mix popup for us to change levels if one is too low or too high. Sliders can be accessed through the `Slider()` class, Declared in the same way as buttons, they have a GUI ID and a name. Sliders also need to be declared with a range and a step value. Here is how to declare a slider:

```
#at the top of the file only
from extronlib.ui import Slider

<slider 1 name> = Slider(<touch panel name>, <GUI ID>)
<slider 2 name> = Slider(<touch panel name>, <Different GUI ID>

<slider 1 name>.SetRange(-18, 24, 0.5)
<slider 2 name>.SetRange(-18, 80, 1)

<slider 1 name>.SetFill(0)
<slider 2 name>.SetFill(20)
```

The `SetRange()` function has three required parameters: lower bound, upper bound, step. Depending on what the slider is used for, these numbers might differ from what we have here. This was just an example of how to declare a slider. The `SetFill()` command sets the slider level to whatever level you want to set the visual feedback to.

6.1.3 Labels

Labels are text objects in the GUI designer that allow us to give the user more nuanced visual feedback. This can range from changing the text box on a number pad with the entered numbers, or to have a piece of text telling the user which project has what source displayed. Labels can be assigned with text dynamically through the `Label()` class. Similar to how button objects are declared, you need a GUI ID and a name. Here is how labels can be declared with a fresh empty string:

```
#Only needed at top of file
from extronlib.ui import Label

<label 1 name> = Label(<touch panel name>, <GUI ID>)
<label 2 name> = Label(<same touch panel>, <Different GUI ID>)

emptystr = ''

<label 1 name>.SetText(emptystr)
<label 2 name>.SetText(emptystr)
```

Labels can be created with any text you want using the `SetText()` method. If no text is set, they will default to what is in GUI Designer. Different labels can use the same string of text but need to be updated any time that string is changed. They also have another method called `SetVisible()` which will either be passed True or False depending on if you want to show or hide the label element in the user interface. Two parameters correspond to these two methods and return whatever the object name in GUI designer, or whether the label is visible. Again, there is no way to get the current text on the label back with a parameter. This is why we always use variables to store strings.

6.2 The Event Decorator

The Event decorator was a cause of a lot of confusion for me when this project began The information on them is difficult to understand in the official documentation so it took a lot of trial and error to get right. The information I used to write this section comes from the ECS Online Training course I took before the EAP

instructor lead sessions. The ECS Online course explained how to use VS Code and some basic instructions for creating a small project. Using that information I created my own methods and expanded the project to match the size and capabilities of our classrooms.

An event defines the response to certain actions. Events are can be used to define what happens when a button on the touch panel is pressed by the user or when a device gets disconnected. You need to have a decorator and then a function definition. The event decorator triggers the function below when the expected response is received.

6.2.1 Using Events

Defining an event requires two things to function. It needs an object name such as a button or slider and it needs a list of states for that button which needs to be defined by the user. The objects provided can be a list of objects or it can be only one object but there needs to be at least one. A function can also have multiple event decorators pointing to it. Here is the structure of a basic event function. It can be expanded to c If you have little to no prior Python experience, there are plenty of free online tutorials containing additional details about the functions briefly covered in this guide. over almost anything:

```
#Basic event structure

#list of button states
ButtonEventList = ['Pressed', 'Released', 'Held', 'Repeated', 'Tapped']

#button and state are parameter names for the function, they can be whatever you want to use
@event(<button name>, ButtonEventList)
def ButtonNamePressed(button, state):
    if state == '<some state in the list>':        #'Pressed' is most common
        button.SetState(<visual feedback state ID>) #default is 1 for on

        #now hide or show popups, or show page
        button.ShowPage('<page name>')

        #maybe you need to check projector state
        if dvPROJ.ReadStatus('Power') == 'Off':
            dvPROJ.SetPower('On', None)

        #maybe you need to swap sources - use .Set() and specify type
        dvScalar.SetInput('<HDMI port number>', {'Type': 'Audio/Video'})

        #other functions that might be needed to do things are found inside communication sheets for
             specific devices like the scalar or the projector.

        if state == '<some other state>':
            #do whatever else you would need
            button.SetState(<different visual feedback state>)
```

Here is an example of an event we wrote inside of the touch panel code to handle the advanced settings page being displayed:

```
#Advanced Settings
btn_advSettings = Button(dvTLP, 47)
@eventEx(btn_advSettings, ['Pressed', 'Released'])
def ShowAdvancedSettingsPopup(button:Button, state):
    print(button.Name, state)
    button.SetState(1 if state is 'Pressed' else 0)
    dvTLP.ShowPopup("Advanced Settings")
```

This is one of the simplest event definitions in the whole code. It only requires one thing to happen, that being the button `btn_advSettings` to be in the state `'Pressed'` which is one of the states in `ButtonEventList` If that happens, the button visual state is set to 1 for 'On' and the popup for advanced settings is shown.

When the button state becomes `'Released'`, the visual state returns to 0 for 'Off'. In this case, other popups are not hidden because the user might have a popup for source selection that would need to be returned to when exiting the advanced settings popup. This structure means in GUI Designer, the advanced settings popup must have a higher layer than any other popup. If it does not have the highest layer, you need a list of the popups the user could return to and assign a variable to keep track of whatever popup they have open. This list would then be used when closing the advanced settings popup to re-open whatever popup was previous. Here is a way to implement the code we just described:

```
#Code to open last popup
popup_list = ['None', 'Popup1', 'Popup2', 'Popup3']
index_num = 0     #setting no popup to 0 means hide all when closing advanced settings

#say user has open Popup1 (index_num = 1) but it's been closed already
#this needs to be kept track of in events that open those popups by doing
#index_num = index('Popup<x>') or whatever the names you have for the currently open popup
#if you want to be fancy you can even use this to open popups where x is the index of whichever
    popup you want open
#dvTLP.ShowPopup(popup_list[x])

@event(btn_advSetClose, ButtonEventList)
def CloseAdvSettings(button, state):
    button.SetState(1 if state is 'Pressed' else 0)
    if state == 'Pressed':
        #something not mentioned is clearing the number pad. It will be mentioned in a later section
        dVTLP.HidePopup('Advanced settings')
        dvTLP.ShowPopup(popup_list[index_num])
```

There are a lot of ways to show and hide popups but the most common is to just write a string with the name in the `.ShowPopup()` function. Pages work the same way but using `.ShowPage()` method instead. The string must be the name in GUI designer. The best practice is to name popups and pages after what they do and to stay consistent so you can look at a code block and know what it does by the names.

Buttons and events can also be used to update labels. This is most commonly used for things like passcode screens. In order to create a passcode you need to have a list of every number button and provide that list to the event decorator. If one of these buttons is pressed, then using the name you can update the string for the label and for the check if the passcode is correct. Here is the code we used to create the main passcode screen with the two options for passcodes (not actually our passcodes) included. This code snippet uses a loop to create the number pad buttons, and various methods to update labels and change strings. It also utilizes something not yet discussed which is the `@eventEx()` decorator. This will be discussed in Section 9 along with the file reading.

```
"""PASSCODE SCREEN"""

passcodeFile = File('user/passcode.txt', 'r')
passcode = str(passcodeFile.readline())

PadButtons = []
for Button_IDs in range(141, 151):
    PadButtons.append(Button(dvTLP, Button_IDs))

LblPadString = Label(dvTLP, 140)
LblString = ''
PadString = ''

def clear_code():
    global PadString
    global LblString
    PadString = ''
    LblString = ''
```

```
19      LblPadString.SetText(LblString)
20
21  @eventEx(PadButtons, ['Pressed', 'Released'])
22  def PadButtonPressed(button:Button, state):
23      button.SetState(1 if state is 'Pressed' else 0)
24      if state is 'Pressed':
25          print(button.Name, state, "Control")
26          global PadString, LblString
27          PadString += button.Name
28          LblString += '*'
29          LblPadString.SetText(LblString)
30
31  #enter and clear
32  btn_passcodeEnter = Button(dvTLP, 152)
33  btn_passcodeClear = Button(dvTLP, 151)
34  btn_passcodeCancel = Button(dvTLP, 153)
35  @eventEx([btn_passcodeEnter, btn_passcodeClear, btn_passcodeCancel], ['Pressed', 'Released'])
36  def BtnEnterPasscode(button:Button, state):
37      print(button.Name, state)
38      global PadString
39      button.SetState(1 if state is 'Pressed' else 0)
40      if (button is btn_passcodeEnter) and ((PadString == '2748') or (PadString == passcode)): #
            whatever the current passcode is
41          print('startup running')
42          dvTLP.ShowPopup('Login')
43          StartupWait = Wait(3, Startup)
44      elif (button is btn_passcodeCancel):
45          dvTLP.ShowPage('Start Page')
46      clear_code()
```

Going line by line we will describe what every important line is doing. Line 3 creates the file object we will be reading the newest passcode from. This uses the Extron `File` class. This opens a file stored directly on the IPCP. This file is read from and converted explicitly to a string into the passcode variable on line 4. Line 6 declares an empty array that we will use for all of the number buttons from GUI designer. Lines 7 and 8 use a loop to create each button, only using 2 lines instead of 10. The range is (141, 151) because there are 10 buttons that need to be created, The button for 1 is ID 141, button 2 is 142, and so on. This is dependent on how you create your button objects in GUI designer but it is best practice to create them in order so their IDs are also in order. It is also best practice to create them with the name corresponding to their number so the `.Name` returns a string with just the button's number. If you were to change the names, you could cast the index of the button to a string and use that instead. Line 8 appends the created button object to the list so at the end there will be a list of 10 button objects. Line 10 creates the Label object and 12 the corresponding string to display the entered numbers. Line 14 defines a function used by enter, cancel, and clear. This function is responsible for clearing the label, and the test string, in order to shorten each function and not repeat code. Line 21 is the event decorator with the list of number button objects so that if any is pressed, this event will be triggered. Line 26 allows the strings to stay consistent across multiple functions by referencing them as a global string and not a local scope variable. This is required otherwise changes made in this function will not be seen by other functions. Line 27 appends the name of whichever button triggered the event to the label string. Line 29 then updates the label so that the user can see that a number was entered. We use asterisks to keep the passcode more secretive. `PadString` is used for comparisons to the correct codes only.

Enter, cancel, and clear are handled by the same event. The comparison against the correct code only happens if the button that was pressed is the enter button. If it was, then the startup function gets called and the login popup is shown. If the cancel button was pressed, then the start page is shown. For all three, the `clear_code` function gets called.

The passcode screens demonstrate the ability to provide user feedback within event calls. User visual feedback is very important for ease of use purposes and so the user understands that something is happening each time they press a button. We also have volume level elements included for things like the computer and

mic audio which the professors are able to adjust on their own. Level sliders are only available to technicians but also provide a feedback of sort to users.

The use of sliders in an event is the same as a button event declaration except for the slider state which is only 'Changed'. The function handler for the event also requires a value parameter which is received from the slider object if a change is made. Here is what a slider declaration and function look like:

```
1 sld_wireless = Slider(dvTLP, 39)
2 sld_wireless.SetRange(-18, 24, 0.5)
3 @event(sld_wireless, 'Changed')
4 def WirelessSlider(slider, state, value):
5     if state == 'Changed':
6         slider.SetFill(value)
7         dvScalar.SetEmbeddedInputGain(value, {'Input': '2'})
```

This slider is the wireless microphone level slider in the small rooms. In the case of this slider it sends the command to the IN1806 switcher. In other rooms with a DSP like the Biamp TesiraFORTE DAN AI the commands to change audio levels will be different, but the slider event will still be defined in the same way. These sliders have not been implemented in the larger room projects.

7 Devices + Utilizing Device Modules

Every system contains a multitude of devices which need to be included in the project. Certain devices have methods to communicate to them, others just need to be plugged in and assigned a port. For example, a projector needs to be connected and declared as an object but a Cynap only needs to be plugged into an HDMI port. Determining what devices need to have control is simple. If you need to affect the devices settings or configuration, control some aspect of its function, or communicate data back and forth from it, then it will need an import file and declaration inside of the `device.py` file. In the smaller rooms, the only device needed as an import outside of the Scalar, is the projector. There are files for all devices we have such as the Wolfvision Document Cameras, but unless the function to turn them on is needed, then the file does not need to be included. In this section we will discuss the devices used in a typical system, how to connect them all to the program, and what needs to be done to get them to work properly as a system.

7.1 Device Integration and Function

In order to distinguish different methods for device connection and what they do, we will place each device into a category which are as follows:

- Devices connected over AV Lan
- Devices controlled using Serial Communication
- Devices without control

Each of these types has a different method of connection and configuration. Below is the drawing we used to create my test system and this document. It has devices that fall into all 3 types listed above and will be discussed in more detail later on.

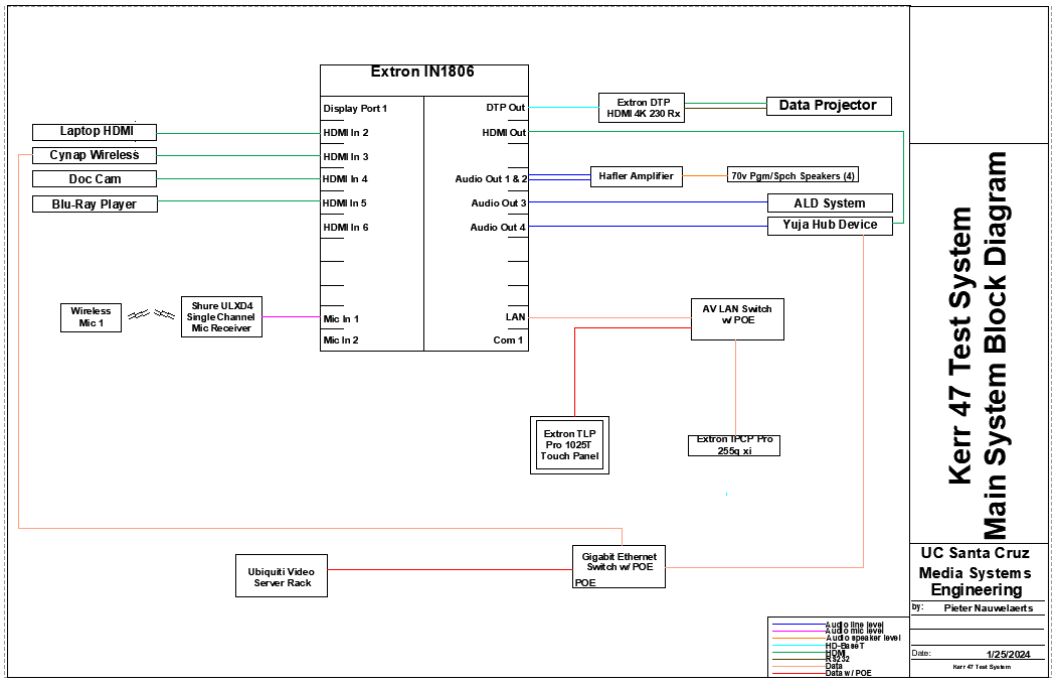

Figure 2: Kerr 47 Test System Diagram

7.1.1 AV Lan Devices

In the test system there are 3 devices connected to an AV Lan switch. The IPCP or the Control Processor, the Switcher or Scalar, and the touch panel. In some systems, a switch is not necessary because the IPCP

is built into the switcher, meaning the cable only needs to go to the touch panel directly from the switcher. These devices need to talk to each other and all need to be included in the code files.

Starting with the IPCP, this is the most important module of the system. When starting a project, the control processor is one of the first devices selected for configuration of the project. This processor is responsible for communication with the switcher and touch panel, as well as control for audio, communication, and building control. In the test system the control processor is not used for any building control, audio, or communication because either there is no need or another device is capable of doing so.

The switcher is where all of the inputs and outputs for audio and video are connected. The IN1806 used in the test system has 5 HDMI and 1 Displayport connections available for use as shown in the drawing. It has 2 line inputs and 4 line outputs as well as DTP out, HDMI out and Serial. In this project, the switcher needs to be connected to over AV Lan which is described in the `devices.py` file. Details on connections to devices will be in a later section where we will go into more detail about how the switcher, projector, or any other devices are connected.

The touch panel is the user interface device which allows a user to select sources, change audio levels, and control the projector. It is connected through POE on the AV Lan switch. The touch panel is what all of the `tlp.py` code controls. The touch panel needs to be included in the json files. The JSON files will be detailed in a later section.

These devices are all assigned IP Addresses when configuring them using Extron's Toolbelt or Extron's PCS. After they are configured, their IPs and product numbers will be used in the JSON.

7.1.2 Serial Devices

There are many devices that might need one or two way serial control. Any device where the touch panel needs to affect internal settings or needs to use a device's specific control functions on the touch panel, most likely needs a serial connection. In the test system, the projector is controlled using Serial over Ethernet. This is a function included in the Extron library which is used in this project to control the device connected on the DTP Out port of the IN1806. The projector is controlled by a DTP receiver.

Another device that could be classified in this section is the Bluray player. They can be controlled through the use of serial communication but to keep this project simple we opted to remove that feature of our touch panel code. If we were to need control over the Bluray we would need to import its device module to access the python functions contained. Then we would define each button like they play, skip, eject, and more, using the included python functions to control the Bluray. we would also need to define the Bluray object inside the `devices.py` with a serial class call.

7.1.3 Non-Controlled devices

Non-controlled devices are any devices not on the AV Lan switch or connected through serial. These devices may have functions to control them, but not any that we need or use in this project. Devices may also need a network connection but not an AV Lan connection. In the test system we have two network switches but in some rooms we may have only one and all campus network devices go straight into a wall network jack. The devices in this category are usually output sources, audio devices like mic receivers or speakers, or external recording devices.

7.2 Defining Devices in Code

In order to control each device using the code, it must be defined as an object within the project. This can be done in one of two ways. The first is in the project configuration file `<Project Name>.json` and the second is in the `device.py` file. Each file has requirements for what can be defined and what is needed to define a device.

7.2.1 JSON Definitions

Devices defined in the project file are usually the control processor and the touch panel. There are a few other devices that can be defined within this file but none are used in any project across the university. Devices defined in this file need a few things:

- Device Name
- Part Number
- Alias
- Network - Host type, LAN and/or AVLAN Addresses
- UI Information (if device is a UI device)

Extron will autofill these fields for you if you type the name of the device, for example if you start typing "IPCP" you can select the device in the dropdown list and it will generate all the required fields for that device type. For IPCP devices, you need to fill in both LAN and AVLAN fields in the network type, even if there isn't a need for a LAN address. If you don't need a LAN address, leave it as the default address from the autofill.

For UI Devices like touch panels, the name of the GUI designer file needs to be included under the `"layout_file"` keyword. This `.gdl` file needs to be in the layout path defined at the top of the project configuration file. The default path is "layout" if following the Extron default template. In the UI section, you can change behaviors such as audio response on actions, or other button feedback.

While the device has now been defined in the configuration file, you still need to create it as an object in the python file. In `devices.py`, each device defined in the JSON needs to be created as an object so that other devices can refer to it or code can affect it. Following are the pieces needed to define a IPCP and touch panel device in the python file:

```
1 from extronlib.device import ProcessorDevice, UIDevice
2
3 #device definitions
4 dvIPCP = ProcessorDevice('<Processor Alias>')
5 dvTLP = UIDevie('<UI Device Alias>')
```

When compiling your project to be deployed, the compiler will look at the JSON file and link the alias names of devices. The objects created can now be used throughout all of the files by importing them. That import looks something like this:

```
1 from devices import dvTLP, dvIPCP
```

Once imported to the file, any methods include in the Extron library for that device can be used in the file. More information about the available commands and what they do can be found in the Control Script Documentation. You can also view more information about the device type by right clicking the imported module and selecting "Go to definition" for a complete list of every function available.

7.2.2 Python Definitions

The python file `device.py` is where the rest of the device definitions are contained. Depending on the device, different connection methods are required to declare the device and connect to it. This file is also where you need to import the Extron device modules, found on the Extron downloads page. These modules contain connection methods, control functions, and a PDF with information about all of the functions. The PDF is called the Communication Sheet and every device module has a sheet to go with it. This sheet has information about connection and supported devices, so make sure to check which method you need to connect your device. Each sheet shows examples of how to connect like in this image:

SerialClass

```
InterfaceName = ModuleName.SerialClass(ProcessorName, 'COM1', Model='EB-PU2010W')
```

SerialOverEthernetClass

```
InterfaceName = ModuleName.SerialOverEthernetClass('192.168.254.254', 2001, Model='EB-PU2010W')
```

EthernetClass

```
InterfaceName = ModuleName.EthernetClass('192.168.254.254', 3629, Model='EB-PU2010W')
```

Figure 3: Device Class Example Definitions

This is the connection methods for an Epson EB-PU2010W, the projector we have in one of the large classrooms. It has three methods of connection. Interface name refers to the name of the projector object. The project this projector was used in has 3 objects, `dvRightPRJ, dvLeftPRJ, dvCenterPRJ`. The module name is from the file import, whatever you import the class as. In my code we call it `Projector` because that's the simplest way to know what each module is responsible for. With VSCode and Intellisense if you hover over the method name, it will give you a popup about what each argument does. For example, hovering over the SerialOverEthernetClass method gives you this list:

- Hostname: DNS Name or IP address of the connection
- IPPort: IP Port number of connection
- Protocol: String value of either 'TCP', 'UDP' or 'SSH'
- Service Port: (UDP only) Port to listen for response data
- Credentials: Username and password of the device, not required
- Model: Model name of device being connected to

Each method has its own requirements for connection so be sure to know what you need to provide the connection method with. Some parameters have default that might be the same as what you use, for example the serial class has default values for Baud, Stop, Parity, and Mode which might match what your device uses to connect. In that case you don't need to provide the class parameters, but if you need to pass a specific one, be sure to pass it by declaring the name because if you skip for example Baud and want to set the number of data bits but instead of saying `.SerialClass(<device>, 'COM1', Data=16,)` you leave it as just 16, then the first available int type parameter will be set to 16.

Here is how we define the devices used in a large classroom:

```
from extronlib.device import ProcessorDevice, UIDevice
from extronlib.system import Timer
from extronlib.interface import EthernetClientInterface

# Project imports
from modules.device import extr_matrix_XTPIICrossPointSeries_v1_12_0_1 as Matrix
from modules.device import biam_dsp_TesiraSeries_v1_15_1_0 as Biamp
from modules.device import tasc_bluray_BD_MP4K_v1_0_0_0 as Bluray
"""Projector is room dependent, this is 150 seat projector"""
from modules.device import epsn_vp_CB_EB_PU100xx_PU2010x_Series_v1_0_2_0 as Projector
"""600 Seat Projector"""
from modules.device import epsn_vp_CB_EB_PU_21xxW_22xxB_Series_v1_0_0_0 as LargeProjector

from modules.helper.ConnectionHandler import GetConnectionHandler
from modules.helper.ModuleSupport import eventEx
from modules.helper.MirrorUI import MirrorUIDevice


# Define devices
dvIPCP = ProcessorDevice('ProcessorAlias')
dvTLPFront = UIDevice('MainPanel')
dvTLPBooth = UIDevice('MirroredPanel')

dvTLPMain = MirrorUIDevice([dvTLPFront, dvTLPBooth])

dvMatrix = Matrix.EthernetClass('10.10.2.30', Model='XTP II CrossPoint 1600')

dvBiamp = Biamp.SSHClass('10.10.2.40', 22, Model='TesiraFORTE DAN AI')

dvBluray = Bluray.EthernetClass('10.10.2.70', 9030, Model='BD-MP4K') #4k in room
```

```
31
32 """150 Seat Projectors"""
33 dvLeftPRJ = Projector.SerialOverEthernetClass('10.10.2.30', 2033, Model='CB-PU2010B')
34 dvCenterPRJ = Projector.SerialOverEthernetClass('10.10.2.30', 2034, Model='CB-PU2010B')
35 dvRightPRJ = Projector.SerialOverEthernetClass('10.10.2.30', 2035, Model='CB-PU2010B')
36
37 """ 600 Seat Projectors - Uncomment when sending to rooms
38 dvLeftPRJ = LargeProjector.SerialOverEthernetClass('10.10.2.30', 2033, Model='CB-PU2010B')
39 dvCenterPRJ = LargeProjector.SerialOverEthernetClass('10.10.2.30', 2034, Model='CB-PU2010B')
40 dvRightPRJ = LargeProjector.SerialOverEthernetClass('10.10.2.30', 2035, Model='CB-PU2010B')
41 """
```

7.3 Device Module Differences

Some device modules behave differently. For example: an Extron device such as a switcher supports the use of `SubscribeStatus` but an Epson projector module may not. In some cases the module will say it does not support `SubscribeStatus` on a particular command, but it will work in deployment. This varies from device to device.

In the deployed code, the best example is the difference between an Epson PU2220B, and an Epson PU2210B. The PU2220B allows the use of `SubscribeStatus` for the `Power` command. It doesn't need a timer event to update the visual response of the On/Off buttons without the need for a timer event to update status. However, the same exact code does not work the same way for the PU2210B. Shown is an example from the PU2220B

```
1  prj_set = MESet([tlp.btn_projOn, tlp.btn_projOff])
2  prj_set.SetCurrent(tlp.btn_projOn if dvCenterPRJ.ReadStatus('Power') is 'On' else tlp.btn_projOff)
3
4  def CenterPowerChanged(command, value, qualifier):
5      GVEServer.SendStatus(PRJC_ID, 'Power', value)
6      if value is 'On' or value is 'Off':
7          prj_set.SetCurrent(tlp.btn_projOn if value == 'On' else tlp.btn_projOff)
8      else:
9          tlp.btn_projOn.SetBlinking('Medium', [0, 1])
10         tlp.btn_projOff.SetBlinking('Medium', [0, 1])
11
12 dvCenterPRJ.SubscribeStatus('Power', None, CenterPowerChanged)
13
14 prj_list = [tlp.btn_projOn, tlp.btn_projOff,
15            tlp.btn_lPrjOn, tlp.btn_lPrjOff,
16            tlp.btn_rPrjOn, tlp.btn_rPrjOff]
17 @eventEx(prj_list, 'Pressed')
18 def ProjectorOnOff(button:Button, state):
19     print(button.Name, button.Host, state)
20     if button in prj_set.Objects:
21         dvCenterPRJ.SetPower('On' if button is tlp.btn_projOn else 'Off', None)
22         dvCenterPRJ.Update('Power')
```

This code does not need a timer, and instead allows the use of `SubscribeStatus` despite the module saying the command is not supported. The PU2210B requires the following code to perform the same actions:

```
1  prj_set = MESet([tlp.btn_projOn, tlp.btn_projOff])
2  prj_set.SetCurrent(tlp.btn_projOn if dvCenterPRJ.ReadStatus('Power') is 'On' else tlp.btn_projOff)
3
4  def CenterPowerChanged(command, value, qualifier):
5      GVEServer.SendStatus(PRJC_ID, 'Power', value)
6      if value is 'On' or value is 'Off':
7          prj_set.SetCurrent(tlp.btn_projOn if value == 'On' else tlp.btn_projOff)
8      else:
```

```
        tlp.btn_projOn.SetBlinking('Medium', [0, 1])
        tlp.btn_projOff.SetBlinking('Medium', [0, 1])
        if PRJCenterTimer.Count == 0: PRJCenterTimer.Restart()
        else: PRJCenterTimer.Resume()

dvCenterPRJ.SubscribeStatus('Power', None, CenterPowerChanged)

def CenterTimer(timer:Timer, count):
    print("Center Timer Count {}".format(count))
    dvCenterPRJ.Update('Power')
    if count > 10:
        timer.Stop()
        print("Center timer stopped")

PRJCenterTimer = Timer(5, CenterTimer)

prj_list = [tlp.btn_projOn, tlp.btn_projOff,
           tlp.btn_lPrjOn, tlp.btn_lPrjOff,
           tlp.btn_rPrjOn, tlp.btn_rPrjOff]
@eventEx(prj_list, 'Pressed')
def ProjectorOnOff(button:Button, state):
    print(button.Name, button.Host, state)
    if button in prj_set.Objects:
        dvCenterPRJ.SetPower('On' if button is tlp.btn_projOn else 'Off', None)
        dvCenterPRJ.Update('Power')
        PRJCenterTimer.Restart()
```

This code requires a timer function to run until a certain point. In this code I use a count included with the Extron timer to decide when to turn off the timer. There are other ways to do this like reading in the current status of the projector. However, the status takes a while to update, so the count is the more consistent method to achieve the desired outcome.

The point of these code snippets is to set the correct visual state for the On/Off buttons on the advanced settings page. The current projector status is needed to set the correct one. If the projector is warming up or cooling down, they both flash slowly, indicating to a user that something is happening if they pressed a source, or one of the buttons.

8 Notes on Version Control

Version control is crucial for managing changes to code or documents, enabling collaboration, tracking revisions, and ensuring the stability of projects over time. This project utilizes UCSC's Gitlab for version control - the repository can be found here. The remainder of this section is for those unfamiliar with Gitlab or Git.

Gitlab is a web-based repository hosting service built on top of the Git version control system. Git is required to use Gitlab, and can be installed here. Those without a UCSC Gitlab account can follow the instructions here to create one. Note that using Git requires the use of the command line, but don't let that scare you!

8.1 Recommended Reading + Resources

- Two minute primer on Git: https://www.youtube.com/watch?v=hwP7WQkmECE
- Git Command-Line for Beginners: https://www.youtube.com/watch?v=HVsySz-h9r4
- Introductory Git Tutorials: https://docs.gitlab.com/ee/tutorials/learn_git.html
- See this guide if you are unfamiliar with how to navigate the command line.

8.2 Git's Core functions: A Quick Reference Guide

Refer to the figure below for a visual representation of Git's workflow.

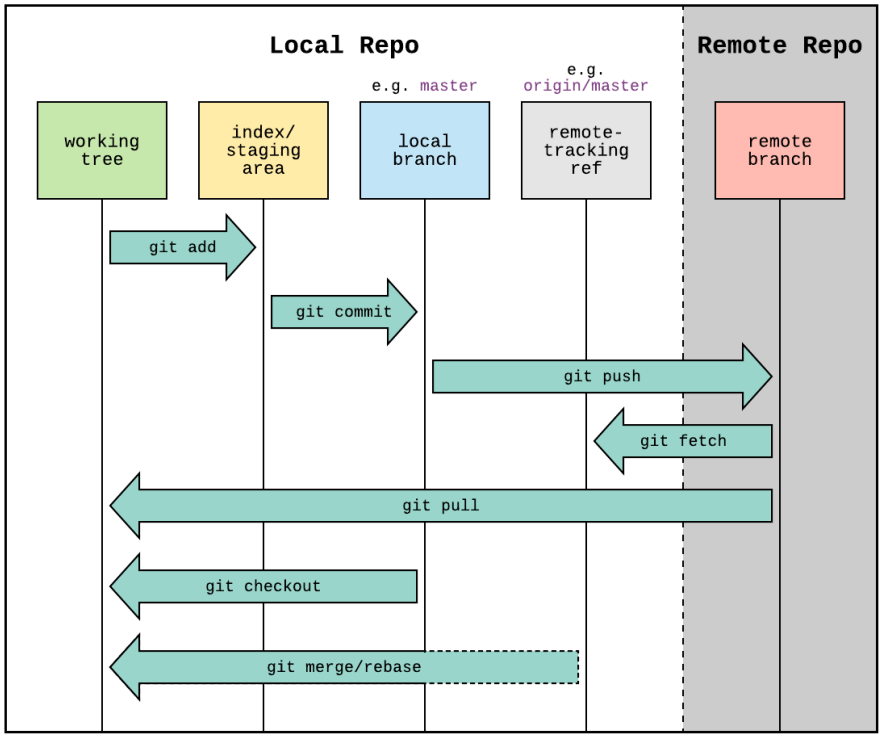

Figure 4: Typical Workflow in Git
From:
https://www.reddit.com/r/git/comments/99ul9f/git_workflow_diagram_showcasing_the_role_of/

8.2.1 Creating a Local Repository

In your command line, change to the directory where you'd like to initialize a repository. Use `git init` to initialize a local repository.

8.2.2 Creating a Remote Repository

To create a new remote repository in Gitlab, navigate to `git.ucsc.edu` and locate the "New Project" button. Fill in all relevant information and create the repository.

To push a local repository to a new remote repository, add and commit (see relevant sections below) the files you'd like the remote repository, then use the following command to simultaneously push and initialize a remote repository.

```
1   git push origin --set-upstream https://git.ucsc.edu/<USERNAME>/<REPOSITORY NAME>.git master
```

8.2.3 Cloning a Repository

In your command line, change your directory to where you would like the repository to be cloned to. In your browser, open the repository on git.ucsc.edu, then obtain the cloning URL (See Figure 5). Generally, cloning with SSH (option a) and HTTPS (option b) should both work, but may require authentication. Use the command `git clone <REPOSITORY CLONING URL>` to clone the repository into the current local directory.

If this repository is one that you'd like to make changes to, add and commit the files you'd like in the repository, then use the following command to set the cloned repository as your remote branch. When you push your commits, the remote repository will reflect whatever changes you've made.

```
1   git push origin --set-upstream <REPOSITORY CLONING URL> master
```

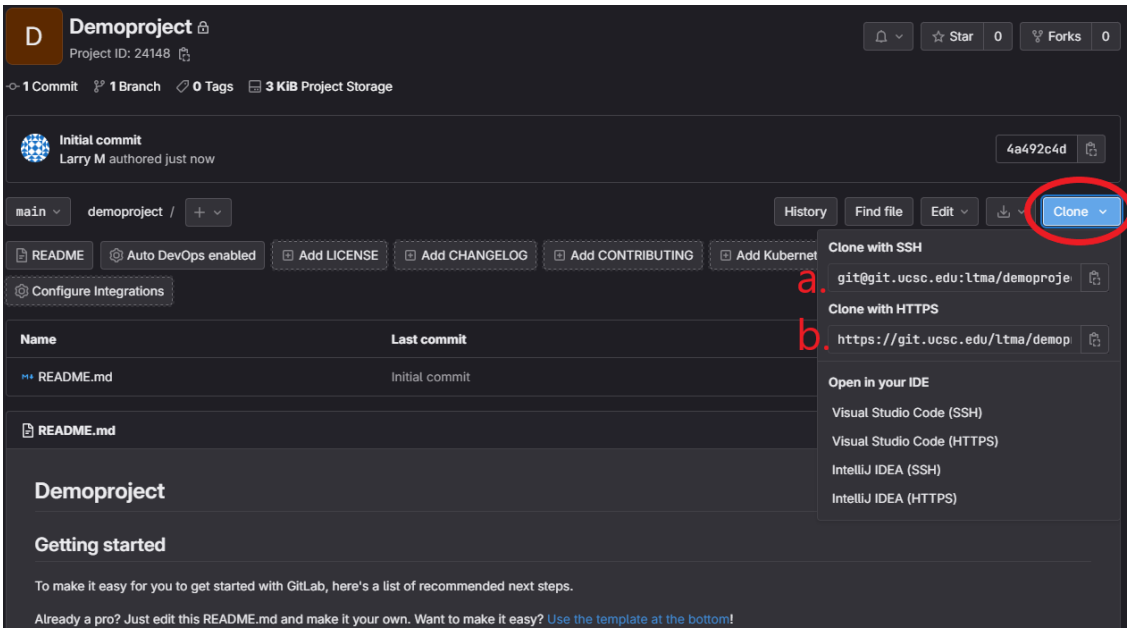

Figure 5: Repository Cloning URL

8.2.4 Pulling Changes

Pulls are made when you want to update your local repository with the the most recent version of the remote repository.

`git pull origin` gets the latest changes from the remote repository, and merges the changes with your local repository.

8.2.5 Pushing Changes

Changes are pushed so that they're saved to the remote repository, where they can be viewed by others. Before you can push your changes, you must first complete two preliminary actions:

1. **Adding files to the staging area**: This is achieved using the `git add <FILES>` command. To avoid having to add files one by one, use the -A flag (`git add -A`) to stage all changes throughout the repository (including deletions and additions), or use `git add .` to stage changes only within the current directory and its subdirectories, excluding deletions.

2. **Commiting the changes**: After staging all relevant files, use `git commit -m "<MEANINGFUL MESSAGE>"` to changes. Committing your files is similar to taking a snapshot of the state of the files at a specific point in time. Make sure to include a message that describes what changes have been made since the last commit or update to the repository.

After completing the above, use `git push origin` to push your changes to the remote repository.

9 Explanation of the Existing Codebase

The existing codebase is split into two Git repositories. The first and most important is the Extron repository where this document is stored. This folder contains all of the code currently deployed or being prepared to deploy for specific rooms. Each folder inside the repo is a project with a name corresponding to the room it will be deployed in. This is the space for completed and working code. Any testing done on new hardware, or test projects to work out functionality issues should be created in the other repository. The second repository is called Extron Development. It is where all of the various test projects and resources are stored. This is where we have Extron example projects, the testbed system code, and old `.gcplus` and `.gdl` files. It also contains the system diagram for the testbed. A lot of the extra Extron resources linked in Section 10 are found in this repo.

9.1 Extron Development Repository

This is the collection point for all of the development resources. Any time we create a project that uses a new structure or different equipment than the other rooms we already wrote code for, we will write it in here first. For example the Westside Research Park system is an NBP system which we had not previously written code for. we started by developing an NBP project in the Development repo before moving the cleaned code to the Extron repository for deployment. This way we keep a copy with all of the notes and information surrounding the equipment type contained in the development repository while the cleaner and easier to read code goes in the deployment repository. This is how we have kept the code base organized but is not necessary if you believe there is a better way to do this.

9.2 Extron Repository

There are a few things in this repository that need to be discussed. In Section 6 the basic idea behind UI objects and Events were discussed. The deployed projects work in mostly the same way as discussed there but with one major exception. `@eventEx()` works in almost the exact same way as `@event()` but with the added bonus of allowing multiple functions to be assigned to the same event. This is required in order for the file divisions to work. Having UI changes handled in the `tlp` files and device control in the `control` files makes the code far easier to read and understand. These file divisions will be discussed in Section 9.3

9.2.1 Small Room Template

The small room type is usually a single projector with an 18 series switcher. They use a 10' table mount touch panel and will in most cases have speech reinforcement. They have a relay device controlling the drawer lock. For the current projects, the projector is controlled by the DTP receiver box using serial over ethernet. They also all contain a BluRay player, controlled over ethernet. Here are the tlp and control files, with what they do:

- `tlp.py` – The start page, help buttons, shutdown buttons, and source selection button definitions. Also is the collection point for every other tlp file, imported at the top.
- `av.py` – The collection point for every other control file, imported at the top. No other control routine defined in this file.
- `tlpAdvanced.py` – The advanced settings popup. Contains the definitions of the projector control buttons and the technician passcode. Includes sliders on tech page.
- `advancedControl.py` – Defines the control routine for the advanced settings projector buttons. This is a special case where visual state is dependent on successful status return from the device. Also includes slider commands
- `tlpBluray.py` – The definition of every button on the BluRay popup.
- `blurayControl.py` – Control for the BluRay popup. Sends commands defined in the BluRay module and the custom state dependent open/close commands

- `tlpMainAudio.py` – The main page audio buttons including the up, down, and mute buttons for both program and speech.
- `mainAudioControl.py` – Main page audio control for mic and program volume. Level feedback is defined in this file to keep it consistent with the device level and not with button presses.
- `shutdownControl.py` – Defines the passcode required at start up. Defines the buttons and imports the file containing the instructor passcode. Defines the shutdown and startup routines including device updates, and setting default values.
- `sourceControl.py` – Source control has its own file that goes with the button definitions in `tlp.py`. This file switches input, and also contains the visual feedback on the laptop popup. It also has the main page video mute control.

These are the most important files of the project along with the `device.py` file with every device definition. In the Extron repository, each file has comments at the top describing the function and some within the code explaining what lines are meant to do. I will not be going in depth about the contents of these files unless they depart significantly from the methods described in Section 6 and Section 7.

When connecting devices to the system, they all need to be activated with the `.Connect()` method. This happens in the `system.py` file. Importing the device file allows you to refer to each device by `devices.<device name>.Connect()`. This is very important otherwise the devices will never be connected to the system and errors will be generated by any commands sent. The connection calls must be made inside the `Initialize()` function for it to be called on startup within the `main.py` file. Main is where Extron connects the ControlScript Deployment Utility to your program, so anything needing to run at initialization should be defined here or called here.

The BluRay is a special case for the device definitions. Because we want to use the `SendAndWait()` command later on in the code, we are unable to use Extron's `GetConnectionHandler()` method. This means having to define the device using a timer and a connection handler event. The definition is as follows:

```
1  #Defines the BluRay as an ethernet device using port 9030 (port specified by Tascam)
2  dvBluray = Bluray.EthernetClass('10.10.2.70', 9030, Model='BD-MP4K')
3
4  #Timer function, called every time the timer runs out
5  def ConnectBluray(timer:Timer, count):
6      #attempt to connect to device, timeout of 5 seconds
7      dvBluray.Connect(5)
8
9  #Define the timer object and callback function, by default stops timer
10 BlurayConnectionTimer = Timer(5, ConnectBluray)
11 BlurayConnectionTimer.Stop()
12
13 #Event on connection or disconnection
14 @eventEx(dvBluray, ['Connected', 'Disconnected'])
15 def BlurayConnectionHandler(client:EthernetClientInterface, state):
16     #Information sent to program log
17     print('Bluray on IP {0} is {1}'.format(client.IPAddress, state))
18     if state is 'Connected':
19         #If the device successfully connected, start the keep alive query, every 30 seconds ask
               BluRay for generic status
20         client.StartKeepAlive(30, '!7?SST\r')
21     else:
22         #If keep alive query was running, stop it and restart the timer. Timer stays attempting
               connection until successful.
23         client.StopKeepAlive()
24         BlurayConnectionTimer.Restart()
```

This device needs to be defined this way in order for the following code to work properly:

```
transport_set = {tlp.btn_blurayStop: 'Stop', tlp.btn_blurayPlay:'Play', tlp.btn_blurayPause: 'Play
    Pause', tlp.btn_blurayRTrack: 'Track Skip Previous', tlp.btn_blurayFTrack: 'Track Skip Next',
    tlp.btn_blurayRewind: 'Rewind', tlp.btn_blurayFForward: 'Fast Forward'}

menu_set = {tlp.btn_blurayDown: 'Down', tlp.btn_blurayRight: 'Right', tlp.btn_blurayUp: 'Up', tlp.
    btn_blurayLeft: 'Left', tlp.btn_blurayEnter: 'Enter', tlp.btn_blurayReturn: 'Return', tlp.
    btn_blurayHome: 'Home', tlp.btn_blurayOption: 'Option Menu', tlp.btn_blurayMenu: 'Setup Menu'}

#Defines an event for every single button on the bluray page.
@eventEx([tlp.btn_blurayStop, tlp.btn_blurayPlay, tlp.btn_blurayPause, tlp.btn_blurayRTrack, tlp.
    btn_blurayFTrack, tlp.btn_blurayDown, tlp.btn_blurayRight, tlp.btn_blurayUp, tlp.btn_blurayLeft
    , tlp.btn_blurayEnter, tlp.btn_blurayReturn, tlp.btn_blurayHome, tlp.btn_blurayOption, tlp.
    btn_blurayMenu, tlp.btn_blurayRewind, tlp.btn_blurayFForward, tlp.btn_blurayEject, tlp.
    btn_blurraySub], 'Pressed')
def TransportEvent(button:tlp.Button, state):
    print(button.Name, state, 'Control')
    #Transport set corresponds to buttons sent using SetTransport('<command>') in the device module
    if button in transport_set:
        dvBluray.SetTransport(transport_set[button], None)
    #Menu set corresponds to buttons sent using SetMenu('<command>') in the device module
    elif button in menu_set:
        dvBluray.SetMenu(menu_set[button], None)
    #The SetSubtitle() command was created by us in our device module. For future projects always
        use a BluRay module included in one of the project folders to have access to the
        SetSubtitle command.
    elif button is tlp.btn_blurraySub:
        dvBluray.SetSubtitle(None, None)
    elif button is tlp.btn_blurayEject:
        #Response contains the device's binary response to the command sent. SendAndWait() sends the
            commmand and waits 2 seconds before continuing.
        response = dvBluray.SendAndWait('!7?MST\r', 2)
        print(response)
        if response:
            #if a response was received, decode it into a string.
            blurayResp = response.decode()
            #TTO is a piece of the command corresponding to if the disk tray is open. If it is, send
                command to close it. The whole response is longer than just TTO but that is the
                only piece that changes depending on state.
            if 'TTO' in blurayResp:
                dvBluray.SetDiskTray('Close', None)
            else:
                dvBluray.SetDiskTray('Open', None)
```

Both the subtitle command and the open or close commands were designed by us. The module only contains the `SetDiskTray('Open')` or `SetDiskTray('Close')` command, but nothing to tell the current state. Relying on the button visual state is also not possible in this case because of the physical button on the device or the remote. States could become out of sync meaning we would be sending the wrong command. To avoid this, before sending the command we designed the routine to wait for the device response. `'!7?MST\r'` is the command to ask the device what the disk tray status is. It can respond in a few ways but we only need to use one of those ways. If the tray is open it will respond with `b'!7MSTTO'`. Checking if `'TTO'` is in the response string allows us to know if we want to close the tray. For any other status we want to open the tray.

The BluRay also uses a Python concept called a dictionary. Dictionaries are key-value pairs that can behave like lists. In this way we are able to check if a key exists by using the `in` keyword. Once we know if the key exists, we can get the command string from the dictionary by using the key as an index. The key in this case is just the button object that triggered the event, so for any button in the menu or transport set. For more information, refer to the python resources in Section 3

The most recent status must be obtained using the `Update('<command>')`. This means the command structure must change if you want to use `SubscribeStatus()`. This is how we handle this specific case, by using a timer to query the device for any changes. It also avoids calling the command constantly and backing up the device with commands it needs to respond to. By calling the command every few seconds rather than every few milliseconds we avoid that backup and keep the program log clear of device replies. We also know that the device takes a few seconds to change status, so there is no need to call it so often.

All other events not described in the previous code snippets work in the same way as in Section 6 using `@eventEx()`. Refer to the code if you need any more examples of how to program two events for the same button in different files. There are comments and more information available in the repository.

For rooms with equipment similar to KRS-3101 or KRS-3301, use these projects as a starting point. The only things you might need to change could be within the GUI layout. You will always need to change the IP address of the controller in the project JSON file. These projects are a good starting point for a TLP 1025T/M, an IN1806/IN1808, and a pair of wireless microphones. The microphone functions are also easy to remove if installing a system in a room without need for speech reinforcement.

9.2.2 Large Room Template

The difference between a small and a large room comes down to the type of switcher used. In the small room case it is often an IN1800 series switcher. We refer to a large room as one with a Matrix switcher. Currently we only have two matrix switchers deployed and they are both XTP CrossPoint II switchers. We do not have any DTP CrossPoint switchers in the rooms but this same project type would apply to those as well. A large room usually comes with a stand alone audio processor, in our case a Biamp TesiraFORTE. They also have a BluRay player, and have 2 to 3 projectors. They will also usually have confidence monitors on the podium.

Matrix switchers use different commands from the IN1800 switchers. Instead of setting the input and having always-on outputs, matrix switchers need an input to be tied to an output with its tie type defined. All of the commands can be found in the command sheet included in each project under `~/src/modules/device`. This will be discussed when I cover the method of source selection in the code.

As with Section 9.2.1, refer to the repository for comments on code or if you are creating a project with the same system requirements as this. There are descriptive comments at the top of every file as well as more information for certain lines of code for why it was defined in such a way.

For the large room project there is one added file: `tlpSourceSelect.py`. It defines the source selection buttons for both center mode and dual mode. Control is defined in the `sourceControl.py` file

Device definitions are all accomplished in the same way as the small room project. For the project specific devices refer to the project containing the device you want to use, and follow that method of connection.

A major difference between the small and large projects, is the method of input switching. For the small room project, buttons correspond to input numbers which are set with the `SetInput('<number>')` command. The large room matrix requires using `SetMatrixTieCommand('<input>', '<output>', '<tie type>')`, specifying which input ties to what output, and what type you want to pass. For example, to output laptop HDMI to the left projector we need to use the command `SetMatrixTieCommand('1', '1', 'Audio/Video')`. To simplify this process we us python global variables defined in the tlp file which allow us to determine which tie commands we need to send. If a button is on the left input set, we know the output needs to be to projector 1. We also again use list indices to know which button corresponds to which input number. Each input needs to be tied to multiple outputs, more specifically 4 outputs; projector, confidence monitor, yuja, and the sound system. Here is the logic in the tlp file:

```
1 #every input button for left right and center is in the total list
2 @eventEx(input_total_list, 'Pressed')
3 def SwitchInput(button:Button, state):
4     #global variable definitions allow them to be used across multiple function and multiple files
5     global prj_select, monitor_select, yuja_select
6     print(button.Name, button.Host, state)
7     dvTLPMain.HideAllPopups()
8     btn_cBoardCams.SetState(0)
9
```

```
    #wireless has an additional popup that needs to be added when initially selected. Every other
        popup is in a list and shows up when button is selected
    if button in [btn_cWireless, btn_lWireless, btn_rWireless]:
        dvTLPMain.ShowPopup('Wireless select device')

    #center set means center projector, left confidence and left yuja (only two yuja inputs and two
        confidence monitors)
    if button in center_input_set.Objects:
        prj_select = 2
        monitor_select = 4
        yuja_select = 9
        dvTLPMain.ShowPopup(popup_list[center_input_set.Objects.index(button)])
        #set visual status
        center_input_set.SetCurrent(button)
    elif button in left_input_set.Objects:
        #set left projector, left yuja, left confidence
        prj_select = 1
        monitor_select = 4
        yuja_select = 9;
        dvTLPMain.ShowPopup(popup_list[left_input_set.Objects.index(button)])
        left_input_set.SetCurrent(button)
        #left source sound becomes active, switch to right source button now visible
        btn_leftSourceSound.SetVisible(False)
        btn_rightSourceSound.SetVisible(True)
    elif button in right_input_set.Objects:
        #set right projector, right monitor, right yuja
        prj_select = 3
        monitor_select = 5
        yuja_select = 10
        dvTLPMain.ShowPopup(popup_list[right_input_set.Objects.index(button)])
        right_input_set.SetCurrent(button)
        #now sound is from right source, set left source sound button to visible
        btn_leftSourceSound.SetVisible(True)
        btn_rightSourceSound.SetVisible(False)
```

This is the logic within the `tlpSourceSelect.py` file. It allows the control file to be simplified in its logic which is shown below:

```
@eventEx([tlp.btn_cBoard1, tlp.btn_cBoard2, tlp.btn_cBoard3], 'Pressed')
def CenterBoardSelectInput(button:Button, state):
    output = tlp.center_board_set.Objects.index(button) + 9
    dvMatrix.SetMatrixTieCommand(None, {'Input': '{}'.format(output), 'Output': '{}'.format(tlp.
        prj_select), 'Tie Type':'Video'})
    dvMatrix.SetMatrixTieCommand(None, {'Input': '{}'.format(output), 'Output': '{}'.format(tlp.
        monitor_select), 'Tie Type': 'Audio/Video'})
    dvMatrix.SetMatrixTieCommand(None, {'Input': '{}'.format(output), 'Output': '{}'.format(tlp.
        yuja_select), 'Tie Type':'Audio/Video'})

@eventEx(tlp.input_total_list, ['Pressed'])
def SourceSelection(button:Button, state):
    print(button.Name, button.Host, state)
    #turn on projector corresponding to whichever mode was selected. Can reduce redundancy by
        comparing to prj_select variable instead of mode variable.
    if tlp.prj_select == 1:
        output = tlp.left_input_set.Objects.index(button) + 1
        dvLeftPRJ.SetPower('On', None)
        PRJLeftTimer.Restart()
    elif tlp.prj_select == 2:
        output = tlp.center_input_set.Objects.index(button) + 1
```

```
18          dvCenterPRJ.SetPower('On', None)
19          dvCenterPRJ.Update('Power')
20          PRJCenterTimer.Restart()
21      elif tlp.prj_select == 3:
22          output = tlp.right_input_set.Objects.index(button) + 1
23          dvRightPRJ.SetPower('On', None)
24          dvRightPRJ.Update('Power')
25          PRJRightTimer.Restart()
26
27      #left and right board cams special case where input value is the same as yuja value (9 for left
            and center, 10 for right). can be hardcoded
28      if button in [tlp.btn_lBoardCams, tlp.btn_rBoardCams]:
29          dvMatrix.SetMatrixTieCommand(None, {'Input': '{}'.format(tlp.yuja_select), 'Output': '{}'.
                format(tlp.prj_select), 'Tie Type': 'Video'})
30          dvMatrix.SetMatrixTieCommand(None, {'Input': '{}'.format(tlp.yuja_select), 'Output': '{}'.
                format(tlp.monitor_select),'Tie Type': 'Audio/Video'})
31          dvMatrix.SetMatrixTieCommand(None, {'Input': '{}'.format(tlp.yuja_select), 'Output': '{}'.
                format(tlp.yuja_select),'Tie Type': 'Audio/Video'})
32          #no tie to output 12 since no audio comes through (reduce error chance)
33      else:
34          #button pressed was not a board camera. This code works for any button left right or center,
                setting the input to all of the correct projectors and outputs needed to be tied.
35          dvMatrix.SetMatrixTieCommand(None, {'Input': '{}'.format(output), 'Output': '{}'.format(tlp.
                prj_select), 'Tie Type':'Video'})
36          dvMatrix.SetMatrixTieCommand(None, {'Input': '{}'.format(output), 'Output': '{}'.format(tlp.
                monitor_select), 'Tie Type': 'Audio/Video'})
37          dvMatrix.SetMatrixTieCommand(None, {'Input': '{}'.format(output), 'Output': '{}'.format(tlp.
                yuja_select), 'Tie Type':'Audio/Video'})
38          dvMatrix.SetMatrixTieCommand(None, {'Input': '{}'.format(output), 'Output': '12', 'Tie Type'
                : 'Audio/Video'})
39          #for error prevention, tie yuja output from BluRay to a blank screen. HDCP content is not
                allowed to Yuja so needs to be not sent.
40          if button in [tlp.btn_cBluray, tlp.btn_lBluray, tlp.btn_rBluray]:
41              dvMatrix.SetMatrixTieCommand(None, {'Input': '0', 'Output': '{}'.format(tlp.yuja_select)
                    , 'Tie Type': 'Audio/Video'})
```

This code structure covers all switching necessary. It also turns on individual projectors based on the buttons pressed. The most important thing allowing it to work is the Python `.format()` command. This command allows us to take a number like the index of a button in a list, use it as a variable, then assign that variable to a string. Because these commands from Extron require using a string, any time we have a variable number like this we need to format it into a string. Python allows us to do that by having the braces within a string. For more information refer to the Python resources in Section 3. Format strings are used several times in the code base such as during the shutdown routine. They make it much easier to code the same functions for different combinations of things like these matrix tie commands.

The large room projects are meant for rooms using matrix style switching. A room without matrix switching, even with more than one projector, should be based on the small room template discussed in Section 9.2.1. However, it must be said that neither of these project types are exact templates. Each project must be changed to match the exact needs of any space. These are the starting points for any similar projects and should be referenced to retain consistency across any deployed code. It is not required to code your projects the way we did ours, but having a good understanding of the way we did ours is necessary in order to solve any errors that may occur, and hopefully will help you to create working projects.

9.3 A Note on File Divisions

Files are split into 3 types in any project developed at this point. These divisions are as follows:

- UI Files: For touch panels or NBP or other user interface files. In our project named `tlp<X>.py` where X can be replaced by a word that describes what it handles such as `tlpPasscode.py`

- Control Files: For device control that is not the touch panel. Any devices defined for control in the `device.py` file will have its code written here. In our project these are named `<X>Control.py` where X describes the functionality.
- Miscellaneous Files: These include the `main.py` file or the `device.py` file. They are starter files provided by Extron which we typically leave named the same. These are the collection point of all the other files and can define startup routines and functions.

Each touch panel file usually handles either a group of objects or one whole page. For example, `tlp.py` defines the startup page button the shutdown buttons, as well as the help buttons and creates the objects for the source selections. The `tlpMainAudio.py` file defines the main page volume and mute buttons. Each name is intended to give us an idea of what kind of buttons go in that file.

The Control files work in a similar way. They each correspond to a tlp file which is imported in the control file. This file is where any device control commands will be sent from. This again is why we need to use `@eventEx()` because it allows us to define both the tlp file and control file function for the same button objects on the same event. This way we have two separate functions controlling the response to a button press. Tlp files usually handle displaying popups or pages, as well as changing the visual state of any button, label, or slider object. There are some exceptions to this rule where the button only changes state on a successful status return from a device, discussed in Section 9.2.1 and Section 9.2.2.

Dividing functionality into separate files makes the project as a whole easier to understand. Files stay shorter and more understandable to read through. This makes writing new functions, or debugging errors far easier. If everything was in one file, finding the error and correcting it would be very difficult with the tools available from Extron. In the current project split, for a given error there are two possible places it could be, either in control or in tlp. Depending on the nature of the error you can narrow it down even further. Add on the fact that the files are far shorter, each being at maximum 200 lines, and the search time for errors goes down further. This is the most important reason why the files are split this way.

10 Additional ControlScript Resources

Below is a list of additional resources on ControlScript Programming. Some are may be more useful than others, but it really depends on what you're looking for...

- ControlScript Framework Documentation https://media.extron.com/public/download/files/articles/ControlScriptFramework_v1x0x0-revL.pdf
- ControlScript Documentation within the ControlScript Extension. An overview of how to use the documentation can be found below (requires an Extron Insider account): https://www.extron.com/kb/help?f=/CSE4VSC/1_0_0/en/Content/HTML_Files/Landing_Page/Using_the_ControlScript_Extension_for_Visual_Studio_Code_Help.htm&hashtag=ControlScriptDoc
- Extron's ControlScript Educational Supplement: https://media.extron.com/public/download/files/progguide/ExtronControlSystemProgrammingUsingPythonB.pdf
- Extron's ControlScript Help File (requires an Extron Insider account): https://www.extron.com/kb/help/CSEVSC
- Extron's ControlScript How-To Videos (requires an Extron Insider account): https://www.extron.com/training/HowToVideos